Builder

Season: 1 · Creational · Category: GoF · Level: Beginner · Build with Waqar

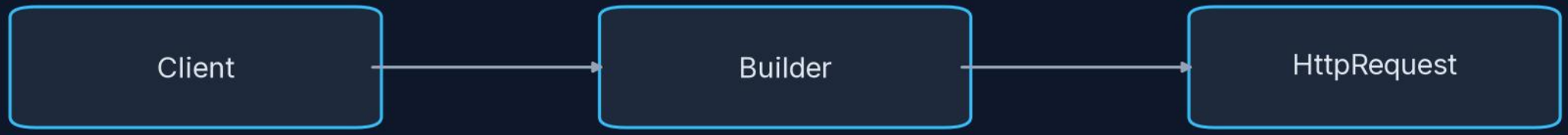
Today's Hook

- Today: Builder as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Builder structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Builder
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/creational/builder/HttpRequest.java
package com.buildwithwaqar.patterns.gof.creational.builder;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

/** Immutable HTTP request built with a fluent classic builder. */
public final class HttpRequest {
    private final String method;
    private final String url;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutMs;

    private HttpRequest(Builder b) {
        this.method = b.method;
        this.url = b.url;
        this.headers = Map.copyOf(b.headers);
        this.body = b.body;
        this.timeoutMs = b.timeoutMs;
    }
```

Implementation (2)

```
    public String method() { return method; }
    public String url() { return url; }
    public Map<String, String> headers() { return headers; }
    public String body() { return body; }
    public int timeoutMs() { return timeoutMs; }

    public static Builder builder(String url) { return new Builder(url); }

    /** Modern: records + withers-style nested builder still works well. */
    public static final class Builder {
        private String method = "GET";
        private final String url;
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body = "";
        private int timeoutMs = 5_000;

        private Builder(String url) {
            this.url = Objects.requireNonNull(url);
        }

        public Builder method(String method) { this.method = method; return this; }
        public Builder header(String k, String v) { headers.put(k, v); return this; }
        public Builder body(String body) { this.body = body; return this; }
        public Builder timeoutMs(int timeoutMs) { this.timeoutMs = timeoutMs; return this; }
```

Implementation (3)

```
        public HttpRequest build() {
            if (timeoutMs <= 0) throw new IllegalArgumentException("timeoutMs must be > 0");
            return new HttpRequest(this);
        }
    }
}

/** Java 25-friendly: compact record with smart constructor + defaults via static factory. */
record ApiCall(String method, String url, String body, int timeoutMs) {
    ApiCall {
        Objects.requireNonNull(method);
        Objects.requireNonNull(url);
        if (timeoutMs <= 0) throw new IllegalArgumentException("timeout");
        body = body == null ? "" : body;
    }

    static ApiCall get(String url) { return new ApiCall("GET", url, "", 5_000); }
    ApiCall withBody(String body) { return new ApiCall(method, url, body, timeoutMs); }
    ApiCall withTimeout(int ms) { return new ApiCall(method, url, body, ms); }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.creational.builder;

public final class Demo {
    public static void main(String[] args) {
        System.out.println("=== Builder walkthrough ===");
        var req = HttpRequest.builder("https://api.buildwithwaqar.dev/v1/videos")
                .method("POST")
                .header("Authorization", "Bearer demo")
                .header("Content-Type", "application/json")
                .body("{\"title\":\"Singleton\"}")
                .timeoutMs(2500)
                .build();
        System.out.println("method=" + req.method());
        System.out.println("url=" + req.url());
        System.out.println("auth=" + req.headers().get("Authorization"));
        System.out.println("timeout=" + req.timeoutMs());
        var call = ApiCall.get("https://x").withBody("hi").withTimeout(100);
        System.out.println("apiCall=" + call.method() + " body=" + call.body() + " t=" + call.timeoutMs());
    }
}
```

Walkthrough Output

```
=== Builder walkthrough ===
method=POST
url=https://api.buildwithwaqar.dev/v1/videos
auth=Bearer demo
timeout=2500
apiCall=GET body=hi t=100
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Builder
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Builder → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.